

大模型内容安全与质量评估智能体 · 项目模块构成

一、项目整体架构

系统有**两条并行实现**，共享同一套核心模块：



二、各模块详解

2.1 入口层

main.py — FastAPI 服务入口

- 端口 8081，挂载 `routes.py` 的路由
- 托管 `static/index.html` 前端页面

- 启动时通过 `python-dotenv` 加载 `.env` 中的环境变量

`app.py` — Streamlit 演示界面

- 提供单条评估和 CSV 批量评估两种模式
- 侧边栏支持选择模型（DeepSeek / Mimo / GPT-4o）和评估 Profile（通用 / 政府公告 / 法律文书）
- 直接复用 `app/` 下的核心模块，不是独立实现

`static/index.html` — 纯 HTML 前端

- 通过 `fetch` 调用 FastAPI 的 12 个 REST 端点
- 展示评估结果：Metric 卡片、瑕疵 DataFrame、Pass-Fail 徽章

2.2 核心引擎

`chain.py` (~1280 行) — 最核心的模块

承载整个评估流程，包含以下功能块：

功能块	函数/类	说明
LLM 工厂	<code>create_llm()</code>	创建单例 <code>ChatOpenAI</code> ，读取 <code>.env</code> 中的 API Key / Base URL / Model
Prompt 构建	<code>build_prompt_messages()</code>	组装 <code>SystemMessage</code> + <code>HumanMessage</code>
对齐模块	<code>build_anchored_text()</code>	将治理前/后文本按段落对齐，加上 <code>[Before N]/[After N]</code> 锚点标记
瑕疵检测	<code>detect_flaws()</code>	基于规则的确定性检测：Markdown 表格/列表结构丢失、段落级删除/新增、字符级数值变化 (>10% 判 critical)
JSON 解析	<code>parse_llm_output()</code>	三层兜底：直接解析 → Markdown 围栏提取 → 正则提取花括号内容
软惩罚	<code>apply_soft_penalty()</code>	取所有瑕疵中最严重的惩罚因子（critical 事实=0.6, critical 结构=0.75），不叠加
裁判判定	<code>determine_verdict()</code>	<code>pass ≥ 0.82</code> , <code>review ≥ 0.5</code> , <code>fail < 0.5</code> （0.78-0.82 缓冲区默认 review）
Profile 惩罚	<code>apply_profile_penalties()</code>	从原文提取关键事实（日期/数字/范围），检查是否在治理后文本中消失
可复现令牌	<code>compute_reproducibility_token()</code>	SHA256(前后文本 + 温度 + 模型 + Prompt版本 + 规则版本 + Profile)
置信度	<code>compute_confidence()</code>	三因子加权：diff 一致性 + reason 完整性 + 规则一致性

功能块	函数/类	说明
风险等级	<code>compute_risk_level()</code>	基于 score + confidence 判定 high / medium / low
自动重采样	<code>_evaluate_once()</code>	评分落在 0.75-0.85 的"临界区"时，自动多跑 2 次取平均，减少误判

核心调用链路：

```
evaluate(request)
  → build_anchored_text()           # 文本对齐 + 锚点标记
  → build_prompt_messages()         # 构建 Prompt
  → create_llm()                     # 获取 LLM 实例
  → llm.invoke()                     # 调用 DeepSeek API
  → parse_llm_output()               # 三层 JSON 解析
  → detect_flaws()                   # 规则引擎瑕疵检测
  → apply_soft_penalty()             # 软惩罚计算
  → determine_verdict()              # 裁判判定
  → apply_profile_penalties()        # Profile 专属惩罚
  → compute_confidence()             # 置信度计算
  → compute_risk_level()             # 风险等级
  → compute_reproducibility_token() # 可复现令牌
  → 返回 EvalResponse
```

engine.py — 向后兼容的薄包装

从 chain.py 重新导出 evaluate、create_llm 等函数，保持旧代码无需修改。

2.3 Prompt 与评估策略

prompts.py — Prompt 模板管理

- **System Prompt**：64 行详细指令，定义 4 个评分维度：

维度	权重	说明
语义一致性	0.35	核心事实、实体名称是否被误改或丢失
事实准确性	0.35	量化指标、日期、数字的精确性
结构完整性	0.15	段落结构、排版、格式是否被破坏
可读性	0.15	语句通顺度、逻辑连贯性

- **Profile 补充规则**：`government_notice_strict` 加强日期/期限/义务敏感度；`legal_strict` 加强条款引用/权利义务准确性
- **Profile Few-Shot**：每种 Profile 附带专门的示例
- **User Prompt 模板**：包含前后文本、段落锚点信息、diff 信息和 JSON 输出格式指令
- `build_system_prompt(profile)` 动态组装：基础 Prompt + Profile 补充 + Few-Shot + 抗偏置指令

profiles.py — 评估 Profile 注册中心

Profile	适用场景	严格点
general	通用文本	默认四维度评分
government_notice_strict	政府公告	日期、期限、阈值、义务范围
legal_strict	法律文书	责任主体、义务、条件、法律后果

每个 Profile 定义 `critical_fact_types`（关键事实类型）和 `penalty_policy`（处罚策略），评估时动态调整 Prompt 和扣分力度。

protocol.py — 抽象评估协议

抽象基类 `EvaluationProtocol`，定义任何模型适配器必须实现的接口：

- `evaluate(request, temperature) -> EvalResponse`
- `get_model_name() -> str`
- `get_model_version() -> str`
- `batch_evaluate(requests, temperature)`（默认顺序执行）

adapters.py — 模型适配器

三个适配器继承 `BaseAdapter`（实现 `EvaluationProtocol`）：

适配器	模型	说明
DeepSeekAdapter	DeepSeek Chat	默认，读 <code>.env</code> 配置
MimoAdapter	小米 Mimo 2.5 Pro	OpenAI 兼容接口
GPTAdapter	GPT-4o	自动追加英文 Prompt 后缀确保 JSON 输出

`BaseAdapter.evaluate()` 的通用流程：构建 Prompt → 调用 LLM → 解析 JSON → 提取维度/瑕疵 → 应用软惩罚 → 判定 verdict → 应用 Profile 惩罚。

model_registry.py — 全局模型注册中心

单例 `ModelRegistry` 类：

- `register(name, adapter, default) / unregister(name)`
- `get(name) → 适配器实例`
- `evaluate(model_name, request) → ProtocolResult`
- `evaluate_all(request) → 所有模型结果字典（用于对比）`
- `register_default_models()` 启动时注册 DeepSeek（默认）、Mimo、GPT（如有 API Key）

2.4 校准与验证

calibration.py — 人机一致性校准

`calibrate(requests, tolerance=0.1) → CalibrationReport:`

对每条样本跑 `evaluate()`，与人工标注对比，计算 5 项指标：

指标	说明
Pearson r	线性相关系数
Spearman ρ	秩相关系数
MAE	平均绝对误差
RMSE	均方根误差
一致率	误差 ≤ 0.1 的样本比例

calibrator.py — 分数校准器

- **ScoreCalibrator**：基于金标准数据集（34 条），对每个模型拟合线性回归（`slope × raw + intercept`），将原始分映射到校准分
- **MultiModelCalibrator**：管理多模型的校准器
- **auto_calibrate()**：在金标准数据上自动拟合

stability.py — 稳定性验证

`run_stability(request, sample_count=5) → StabilityReport:`

- 对同一输入跑 N 次（`temperature=0.0`），计算每维度的均值、方差、标准差
- **瑕疵投票机制**：按 (类别, 片段前30字符) 分组，多数投票决定严重程度；三票平局则降级
- 方差来源诊断：LLM 波动 / 规则放大 / 稳定
- 判定标准：方差 < 0.005 为稳定

2.5 指标与偏置

metrics.py — 评估指标计算

指标	函数	说明
瑕疵 P/R/F1	<code>compute_flaw_metrics()</code>	贪心一对一匹配（按类别）
锚点准确率	<code>compute_anchor_accuracy()</code>	snippet 重叠（ ≥ 4 字符）+ start_char 容差
Cohen's Kappa	<code>compute_kappa()</code>	二分类(pass/fail) 或三分类(pass/review/fail)
Kendall's W	<code>compute_kendalls_w()</code>	简化为 $(1 + \text{Spearman } \rho)/2$
ROUGE-L	<code>compute_rouge_l()</code>	基于 LCS 的文本相似度（纯 Python 实现）
BERTScore	<code>compute_bertscore()</code>	基于 BERT 词向量的语义相似度（可选）

debias.py — 抗偏置模块

功能	函数	说明
长度偏置检测	<code>detect_length_bias()</code>	前后文本长度比，<0.3 高风险，<0.6 或 >2.0 中风险
位置偏置检测	<code>detect_position_bias()</code>	瑕疵是否集中在前/后半部分 (>80% = 偏置)
偏置缓解得分	<code>compute_bias_mitigation_score()</code>	维度分数标准差 × 0.5 + 长度因子 × 0.5
抗偏置 Prompt 指令	<code>generate_anti_bias_prompt_supplement()</code>	长度无关/位置无关/格式无关/领域无关，自动追加到 System Prompt

2.6 报告与存储

reporter.py — 综合报告生成器

`run_full_evaluation(requests)` 串联 7 大模块，生成完整验收报告：

1. 评估

→ `engine.evaluate()`

逐条评分

2. 校准

→ `calibration.calibrate()`

Pearson/Spearman/MAE/RMSE

3. 稳定性

→ `stability.run_stability()`

多次采样方差

4. 瑕疵指标

→ `metrics.compute_flaw_metrics()`

P/R/F1

5. 锚点准确率

→ `metrics.compute_anchor_accuracy()`

字符容差匹配

6. 偏置分析

→ `debias` 模块

长度/位置偏置检测

7. 可复现性

→ `reproducibility_token`

SHA256 令牌验证

↓

I1-I8 验收标准逐项检查

storage.py — 历史持久化

JSONL 追加写入 (`data/eval_history.jsonl`):

- `save_evaluation(response)`: 追加一条记录
- `load_history(request_id, limit, offset)`: 分页查询
- `find_by_token(token)`: 按可复现令牌查找
- `update_human_verdict(request_id, human_verdict)`: 更新人工判定
- `history_stats()`: 汇总统计

batch.py — 批量评估

- `asyncio.Semaphore(5)` 并发控制
- SHA256 内存缓存 (256 条上限)，相同输入直接返回
- 指数退避重试 (1s → 2s → 4s, 最多 3 次)
- 进度回调: 每条评估完成后调用 `callback` 报告进度
- 同步包装 `batch_evaluate()` 供非异步上下文使用

2.7 数据模型

models.py — 所有 Pydantic v2 模型

模型	用途
EvalRequest	输入：前后文本 + Profile + human_label + stabilize 标志
EvalResponse	输出：维度分数 + 总分 + 瑕疵列表 + verdict + 令牌 + 置信度 + 风险等级
FlawItem	单条瑕疵：类别 / 严重程度 / 描述 / 位置 / 建议
AnchorSpan	瑕疵位置：segment_id + 字符范围 + 片段
DimensionScore	单维度分数 + 权重 + 理由
StabilityReport	稳定性分析结果
CalibrationReport	校准指标集

2.8 路由层

routes.py — FastAPI 路由（12 个端点）

端点	方法	说明
/api/v1/evaluate	POST	单条评估（支持 stabilize 模式）
/api/v1/stability	POST	多次采样稳定性分析
/api/v1/calibrate	POST	人机一致性校准
/api/v1/metrics/flaw-detection	POST	瑕疵检出 P/R/F1
/api/v1/metrics/anchor-accuracy	POST	锚点定位准确率
/api/v1/debias/detect	POST	偏置检测
/api/v1/report/full	POST	综合验收报告
/api/v1/batch/evaluate	POST	批量评估（上限 50 条）
/api/v1/history	GET	评估历史查询
/api/v1/history/stats	GET	历史统计
/api/v1/reproduce/{token}	GET	按令牌复现结果
/api/v1/review/{request_id}	POST	更新人工判定
/api/v1/cache/stats	GET	缓存统计
/api/v1/cache/clear	DELETE	清除缓存
/api/v1/health	GET	健康检查

三、测试

文件	类型	说明
tests/test_evaluate.py	独立脚本	加载 eval_dataset，跑 live API，输出校准报告
tests/test_evaluation_profiles.py	pytest	7 个测试，验证 Profile 配置，无需 API
tests/test_day6_minimal.py	unittest	6 个测试，用 FastAPI TestClient 测端点

四、数据文件

文件	说明
data/eval_dataset.json	评估数据集（~34 条，含人工标注的 overall_score 和 flaws）
data/gold_dataset_v1.json	金标准数据集（34 条，用于校准器线性回归拟合）
data/eval_history.jsonl	评估历史日志（追加写入，JSONL 格式）

五、模块依赖关系

```
routes.py
  → engine.py → chain.py（核心逻辑）
  → calibration.py → engine.py
  → stability.py → engine.py
  → metrics.py（独立）
  → debias.py（独立）
  → reporter.py → engine.py, calibration.py, metrics.py, stability.py, debias.py
  → storage.py（独立）
  → batch.py → engine.py, storage.py

chain.py
  → models.py
  → profiles.py（无依赖）
  → prompts.py → debias.py, profiles.py
  → LangChain（ChatOpenAI, ChatPromptTemplate, callbacks, caches）

adapters.py
  → protocol.py → models.py
  → chain.py（parse_llm_output, extract_dimensions 等）
  → prompts.py

model_registry.py
  → protocol.py
  → adapters.py

calibrator.py
  → models.py
  → scipy.stats

app.py（Streamlit）
```



```
→ engine.py, models.py, profiles.py, stability.py, batch.py, storage.py,  
model_registry.py, adapters.py
```

六、一句话总结

系统以 DeepSeek 为裁判引擎，通过精心设计的四维度评分 Prompt + 规则引擎双通道瑕疵检测，实现了对数据治理前后文本的自动化质量评估。核心架构采用 LangChain LCEL 管线，支持多模型适配（DeepSeek / Mimo / GPT）、三种评估 Profile、抗偏置机制、稳定性验证和分数校准，最终通过综合报告生成器输出 I1-I8 验收结果。